

Evaluating a From-Scratch PatchCore on MVTec AD: Metric Protocol, Seed Variance, False-Alarm Intervals and Threshold Calibration

Aghasalim Mustafazada
Howest University of Applied Sciences, Kortrijk, Belgium
`salim.mustafazada@student.howest.be`

Murad Eyvazov
University of Massachusetts Boston, Boston, USA

Abstract

Visual anomaly detectors for industrial inspection are usually compared on a single number, the mean image AUROC over the MVTec AD categories. That number says nothing about how the pixel metrics were computed, how much the result moves with the random seed, or whether a deployable threshold keeps its promised false-alarm rate. We reimplement PatchCore from scratch and evaluate it on all 15 MVTec AD categories with these questions in mind. First, computing pixel AUROC and AUPRO with the standard protocol, over the whole test split and with the exact PRO curve, raises the mean pixel AUROC from 0.9684 to 0.9766 and the mean AUPRO from 0.8457 to 0.9025 compared with our earlier non-standard computation. Second, over five seeds on one GPU, the mean image AUROC is 0.9873 ± 0.0016 , with `screw` the least stable category at 0.9438 ± 0.0175 . Third, at a threshold calibrated on training normals for a 1% target, the pooled false-alarm rate on test normals is 7 of 467 (1.5%, Wilson 95% interval 0.7% to 3.1%), but single categories have only 12 to 60 test normals and their intervals are wide. Fourth, a 99th-percentile threshold on a single 10% holdout meets the 1% target in 3 of 15 categories; 5-fold calibration raises this to 10 of 15 and a nonparametric tolerance bound to 13 of 15, at a cost in recall. Fifth, anomalib’s PatchCore, scored by the same metric code, agrees with ours to within 0.003 on every mean metric. We also report a device effect on the threshold and a localisation check against a random-map control.

1 Introduction

Anomaly detection on MVTec AD [2, 3] is now a mature benchmark. PatchCore [9] reaches a mean image AUROC close to 0.99 with a frozen backbone and no training loop, and libraries such as anomalib [1] make it easy to run. Because the headline numbers are so close

to 1, small differences in evaluation code can matter as much as differences between methods. A reader who sees a mean image AUROC also cannot tell how the system would behave on a production line, where it must answer OK or DEFECT at a fixed threshold chosen without any defect labels.

This paper is a reproducibility and evaluation study. We do not propose a new detector. We reimplement PatchCore in a public repository [8] and measure the parts of its evaluation that a single AUROC leaves out. All numbers in this paper are read from result files committed to that repository. Our contributions are:

1. **Metric protocol.** We show how an earlier, non-standard computation of pixel AUROC and AUPRO understated both, and report the values under the MVTec AD protocol (Section 5.1).
2. **Seed variance.** We run all 15 categories with five seeds on one NVIDIA RTX A5000 and report the mean and sample standard deviation per category (Section 5.3).
3. **False alarms with intervals.** We report the false-alarm rate on test normals at a deployable threshold, with Wilson 95% intervals [13], and show why the small number of test normals makes these intervals wide (Section 5.4).
4. **Threshold calibration.** We reproduce a bug in a 99th-percentile threshold taken from a single holdout and compare it with 5-fold calibration and a nonparametric tolerance bound, on real data and on synthetic scores with a known distribution (Section 5.5).
5. **Same-pipeline comparison.** We run anomalib 2.7.0’s PatchCore through the same images, preprocessing, masks and metric code (Section 5.6).
6. **Device note.** We show that the same script with the same seed gives a different threshold and a

different false-alarm count on Apple MPS and on CUDA (Section 5.7).

7. **Localisation.** We score where the anomaly map points, against a random-map control, and compare with Grad-CAM on a supervised classifier (Section 5.2).

2 Related work

Memory-bank detectors. SPADE [4] scores test features by their distance to features of normal training images. PaDiM [5] fits a Gaussian per patch position. PatchCore [9] stores locally aggregated mid-level patch features of a frozen WideResNet50-2 [14] (a wide variant of ResNet [6]) in a memory bank, reduces the bank with a greedy k-center coreset [11], and scores each test patch by its nearest-neighbour distance. We follow PatchCore, with the deviations listed in Section 3.

Benchmark and metrics. MVTec AD [2] has 15 categories of objects and textures, with only normal images in the training split and pixel masks for every defective test image. The journal version [3] describes the per-region overlap (PRO) metric and its area up to a false-positive rate of 0.3, which we call AUPRO. MVTec AD 2 [7] adds harder scenarios. We do not evaluate on it.

Libraries. anomalib [1] collects many anomaly detection models, including PatchCore, with shared training and evaluation code. We use only its PatchCore model and replace everything around it with our own code, so that the comparison isolates the model.

Thresholds and intervals. We use the Wilson score interval [13] for binomial proportions and a one-sided distribution-free tolerance bound based on order statistics [12]. For explanation baselines we use Grad-CAM [10].

3 Method

Features. A frozen ImageNet WideResNet50-2 from torchvision maps each image to its `layer2` and `layer3` feature maps. Each map is average-pooled with a 3×3 window and stride 1, `layer3` is bilinearly upsampled to the grid of `layer2`, and the two are concatenated. At an input size of 224 this gives a 28×28 grid of 1536-dimensional patch features.

Memory bank. All patches of all normal training images form the bank. A greedy k-center coreset keeps 1% of them. The distance search runs in a 128-dimensional Gaussian random projection, while the bank stores the full 1536-dimensional vectors. The seed sets the projection and the first selected point.

Scoring. Each test patch is scored by its Euclidean distance to the nearest bank patch. The 28×28 score grid is bilinearly upsampled to 224×224 and smoothed with a Gaussian of $\sigma = 4$ using reflect padding. The image score is the maximum patch distance.

Deviation from the paper. PatchCore reweights the maximum patch distance by how isolated the matched bank point is. Our implementation leaves this reweighting out and uses the plain maximum. Section 5.6 compares against an implementation that includes it.

Preprocessing. Images are resized to 256 and centre-cropped to 224, as in the paper, and normalised with ImageNet statistics. Masks receive the same transform.

4 Evaluation protocol

Data. All 15 MVTec AD categories. The official download was not available to us, so the images come from a mirror; the repository’s fetch script fails if any defective image lacks its mask. Training splits hold 60 (toothbrush) to 391 (hazelnut) normal images, and test splits hold 12 to 60 normal images per category, 467 in total.

Image metric. Image AUROC over the full test split.

Pixel metrics. Pixel AUROC is computed over every pixel of every test image, normal images included. AUPRO follows the MVTec AD evaluation [3]: regions are the 8-connected components of each ground-truth mask, each region has equal weight, the false-positive rate counts every defect-free pixel of the whole test split, the curve has one point per distinct score, and it is integrated with the trapezoid rule up to a false-positive rate of 0.3, with linear interpolation at 0.3, and divided by 0.3.

Localisation metrics. On defective test images only, *peak-in-mask* is the share of images whose highest-scoring pixel lies inside the ground-truth mask, and *top-1% precision* is the share of the 1% highest-scoring pixels that are defect pixels. Every localisation metric is also computed for a random map with no spatial information on the same images, as a control.

Threshold rules. A threshold must come from normal training images only. We compare three rules, all with a 1% false-alarm target:

- *Old rule.* Hold out 10% of the training images, build the bank from the rest, and take the 99th percentile of the holdout image scores.

- *5-fold, 99th percentile.* Split the training images into 5 folds. Score each fold against a bank built from the other four, so that every training image gets one out-of-bank score, and take the 99th percentile of these scores.
- *5-fold, tolerance bound (shipped).* The same calibration scores, read off as a one-sided nonparametric tolerance bound with 99% coverage and 95% confidence. For the m -th smallest of n scores, the coverage $F(X_{(m)})$ follows a $\text{Beta}(m, n - m + 1)$ distribution, and the rule takes the smallest m with $P(\text{Beta}(m, n - m + 1) > 0.99) \geq 0.95$. Such an m exists only if $1 - 0.99^n \geq 0.95$, that is $n \geq 299$. For smaller n the rule falls back to the sample maximum and records that the guarantee is not met.

For the 5-fold rules the shipped bank is built from all training images.

False-alarm rate. The false-alarm rate (FAR) is the share of test normals with a score at or above the threshold. For k false alarms out of n normals we report the Wilson score interval at 95% [13]. The repository checks this interval against R’s `prop.test` without continuity correction.

Seeds and devices. The backbone and the test images are fixed, so the seed changes only the coreset and the assignment of training images to calibration folds. The single-seed results (seed 0) were run on an Apple M4 laptop GPU through PyTorch’s MPS backend. The five-seed results (seeds 0 to 4) and the anomalib comparison were run on one NVIDIA RTX A5000 with CUDA. We report the two sets separately.

5 Results

5.1 Pixel metrics under the standard protocol

An earlier version of our evaluation computed pixel AUROC and AUPRO on the defective test images only, and computed AUPRO on a 64-step threshold grid. Both differ from the MVTec AD protocol. Leaving out the normal test images removes many defect-free pixels from the false-positive count, and a coarse grid approximates the PRO curve. Table 1 shows the effect on the 15-category means (seed 0, MPS). Image AUROC and peak-in-mask did not change, since neither depends on these choices.

To check the new AUPRO code, we compared it with anomalib 2.7.0’s AUPRO on four random synthetic cases. The two agree to five decimals in every case. In the first case both give 0.81914, while the old code gives 0.7963.

Table 1: Effect of the pixel-metric protocol on the 15-category means (seed 0, MPS): the non-standard computation (before) and the MVTec AD protocol (after). The published PatchCore value is the one recorded in our repository.

	before	after	published
mean pixel AUROC	0.9684	0.9766	0.981
mean AUPRO	0.8457	0.9025	n/a

5.2 Single-seed reproduction and localisation

Table 2 lists the single-seed results. The mean image AUROC is 0.9874, against 0.990 recorded for the paper. The largest gap is **screw**, 0.9412 against 0.981. **grid** (0.9699 against 0.981) and **pill** (0.9569 against 0.966) are the next largest.

Detection and localisation are different problems. The peak of the anomaly map falls inside the defect for 81.4% of defective images on average, and the measured rate is above the random-map control in every category (Figure 1, left). The spread is large, however. **toothbrush** has an image AUROC of 1.0000 but a peak-in-mask rate of 0.5667. **screw**, whose defects cover 0.43% of the image, has the lowest rate, 0.4958, against a control of 0.0000. Pixel AUROC hides this: **screw** still has a pixel AUROC of 0.9716, because the metric is dominated by easy background pixels.

Supervised baseline. For three categories we trained a ResNet18 classifier on half of the defective test images (plus the normal training images) and compared it with PatchCore on the other, held-out half. Table 3 shows that the classifier detects about as well as PatchCore, but its Grad-CAM maps localise much worse. On **screw**, the classifier reaches an image AUROC of 0.9586 while its Grad-CAM peak never falls inside a defect. This run predates the pixel-metric fix, and its localisation metrics use the defective held-out images only, so we report only image AUROC and peak-in-mask.

Ablations on screw. Earlier single-seed runs on **screw** without the centre crop show where the image AUROC comes from. At the same bank size of 2508 patches, random sampling gives 0.5518 and the greedy coreset gives 0.8737. Growing the coreset to 5% and 10% gives 0.9182 and 0.9289. Adding the paper’s centre crop at 1% gives 0.9412. These runs predate the pixel-metric fix, so we use only their image AUROC.

5.3 Seed variance on CUDA

Table 4 reports seeds 0 to 4 on CUDA. For the mean row, we first average over categories for each seed and then take the mean and sample standard deviation over

Table 2: Single-seed results (seed 0, MPS), 1% coreset. “Paper” columns are the PatchCore values recorded in the repository. PiM is peak-in-mask, ctrl is the random-map control on the same images, top-1% is top-1% precision, and defect px is the fraction of defect pixels in the defective test images.

category	img AUROC	paper	px AUROC	paper	AUPRO	PiM	PiM ctrl	top-1%	defect px
bottle	1.0000	1.000	0.9830	0.986	0.9295	0.9841	0.0952	0.9398	0.0991
cable	0.9983	0.993	0.9841	0.984	0.9255	0.9239	0.1087	0.8321	0.0611
capsule	0.9773	0.980	0.9863	0.988	0.9172	0.6881	0.0183	0.4258	0.0145
carpet	0.9904	0.987	0.9880	0.990	0.9333	0.8090	0.0112	0.6892	0.0269
grid	0.9699	0.981	0.9689	0.987	0.8830	0.6316	0.0175	0.4235	0.0114
hazelnut	1.0000	1.000	0.9841	0.987	0.9355	0.8571	0.0571	0.7267	0.0437
leather	1.0000	1.000	0.9907	0.993	0.9582	0.8913	0.0109	0.5161	0.0111
metal_nut	0.9990	0.998	0.9853	0.984	0.9194	0.9462	0.1935	0.8757	0.1884
pill	0.9569	0.966	0.9741	0.976	0.9267	0.6950	0.0355	0.5690	0.0519
screw	0.9412	0.981	0.9716	0.994	0.8911	0.4958	0.0000	0.2653	0.0043
tile	0.9917	0.987	0.9554	0.959	0.7937	0.9048	0.1429	0.8350	0.1153
toothbrush	1.0000	1.000	0.9842	0.987	0.8187	0.5667	0.0000	0.4955	0.0272
transistor	1.0000	1.000	0.9710	0.964	0.9458	0.9500	0.1500	0.7173	0.1552
wood	0.9895	0.992	0.9393	0.951	0.8307	0.9000	0.1000	0.7218	0.0600
zipper	0.9968	0.985	0.9829	0.989	0.9298	0.9664	0.0420	0.8679	0.0328
mean	0.9874	0.990	0.9766	0.981	0.9025	0.8140		0.6601	

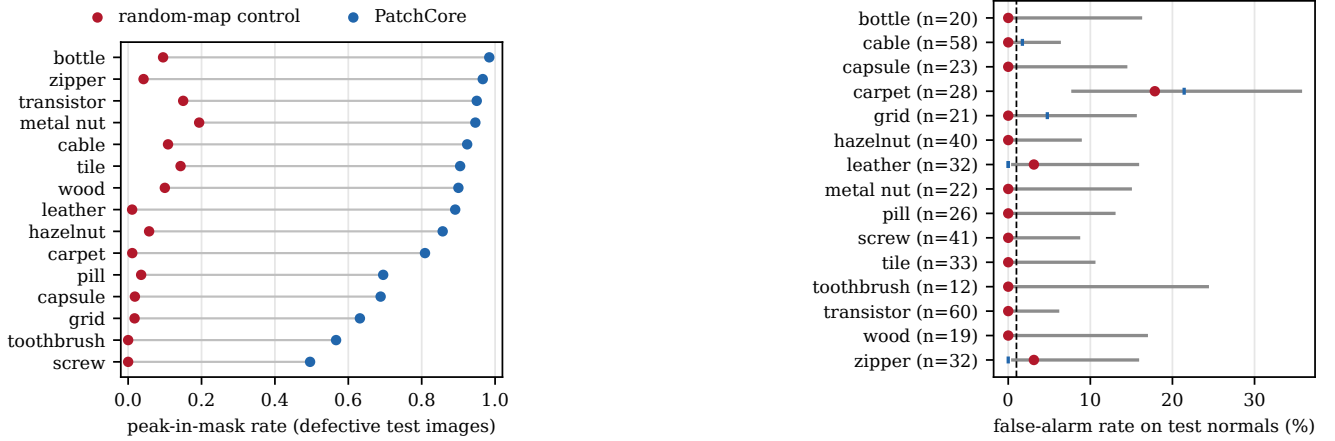


Figure 1: Left: peak-in-mask rate per category (seed 0, MPS) against a random map on the same images. Right: false-alarm rate on test normals at the shipped threshold (CUDA). Red dots are seed 0, grey bars its Wilson 95% interval, blue ticks the five seeds, and the dashed line the 1% target. n is the number of test normals.

Table 3: Supervised ResNet18 with Grad-CAM against PatchCore on the same held-out half of the test split (n images, single run).

category	n	image AUROC		peak-in-mask	
		classifier	PatchCore	Grad-CAM	PatchCore
bottle	42	1.0000	1.0000	0.6875	0.9688
pill	86	0.9526	0.9579	0.3562	0.6712
screw	82	0.9586	0.9375	0.0000	0.4918

seeds. The mean image AUROC is 0.9873 ± 0.0016 , the mean pixel AUROC 0.9775 ± 0.0003 and the mean AUPRO 0.9041 ± 0.0005 . **screw** varies the most in image AUROC (± 0.0175), followed by **grid** (± 0.0078) and **capsule** (± 0.0072). For AUPRO the largest spread is

toothbrush (± 0.0149). For these categories, single-seed differences of a few thousandths in image AUROC should not be read as real.

5.4 False alarms at a deployable threshold

Table 5 gives the false-alarm rate on test normals at the shipped threshold (5-fold calibration, tolerance bound, 1% target) for the CUDA runs. Pooled over all 15 categories, seed 0 flags 7 of 467 test normals, 1.5%, with a Wilson 95% interval of 0.7% to 3.1%. The pooled rate is close to the target, but this hides one category. **carpet** flags 5 of 28 normals at seed 0 (17.9%, interval 7.9% to 35.6%), and between 17.9% and 21.4% over the five

Table 4: Five seeds (0 to 4) on one NVIDIA RTX A5000, 1% coresets. Mean $\pm$ sample standard deviation over seeds.

category	image AUROC	pixel AUROC	AUPRO
bottle	1.0000 $\pm$ 0.0000	0.9831 $\pm$ 0.0001	0.9291 $\pm$ 0.0018
cable	0.9968 $\pm$ 0.0021	0.9841 $\pm$ 0.0003	0.9244 $\pm$ 0.0019
capsule	0.9777 $\pm$ 0.0072	0.9867 $\pm$ 0.0003	0.9075 $\pm$ 0.0059
carpet	0.9887 $\pm$ 0.0015	0.9878 $\pm$ 0.0001	0.9313 $\pm$ 0.0019
grid	0.9781 $\pm$ 0.0078	0.9700 $\pm$ 0.0021	0.8780 $\pm$ 0.0077
hazelnut	1.0000 $\pm$ 0.0000	0.9844 $\pm$ 0.0003	0.9365 $\pm$ 0.0037
leather	1.0000 $\pm$ 0.0000	0.9906 $\pm$ 0.0001	0.9584 $\pm$ 0.0007
metal_nut	0.9988 $\pm$ 0.0010	0.9844 $\pm$ 0.0006	0.9157 $\pm$ 0.0023
pill	0.9541 $\pm$ 0.0032	0.9766 $\pm$ 0.0017	0.9284 $\pm$ 0.0015
screw	0.9438 $\pm$ 0.0175	0.9786 $\pm$ 0.0028	0.9060 $\pm$ 0.0098
tile	0.9879 $\pm$ 0.0017	0.9552 $\pm$ 0.0007	0.7953 $\pm$ 0.0032
toothbrush	0.9967 $\pm$ 0.0050	0.9856 $\pm$ 0.0008	0.8423 $\pm$ 0.0149
transistor	0.9997 $\pm$ 0.0007	0.9732 $\pm$ 0.0030	0.9463 $\pm$ 0.0016
wood	0.9912 $\pm$ 0.0018	0.9398 $\pm$ 0.0014	0.8344 $\pm$ 0.0012
zipper	0.9960 $\pm$ 0.0008	0.9826 $\pm$ 0.0001	0.9282 $\pm$ 0.0009
mean	0.9873 $\pm$ 0.0016	0.9775 $\pm$ 0.0003	0.9041 $\pm$ 0.0005

Table 5: False alarms (FA) among the n test normals at the shipped threshold, CUDA. Seed 0 with its Wilson 95% interval, and the range of the rate over seeds 0 to 4.

category	n	FA	FAR	Wilson 95%	over seeds
bottle	20	0	0.0%	[0.0%, 16.1%]	0.0% to 0.0%
cable	58	0	0.0%	[0.0%, 6.2%]	0.0% to 1.7%
capsule	23	0	0.0%	[0.0%, 14.3%]	0.0% to 0.0%
carpet	28	5	17.9%	[7.9%, 35.6%]	17.9% to 21.4%
grid	21	0	0.0%	[0.0%, 15.5%]	0.0% to 4.8%
hazelnut	40	0	0.0%	[0.0%, 8.8%]	0.0% to 0.0%
leather	32	1	3.1%	[0.6%, 15.7%]	0.0% to 3.1%
metal_nut	22	0	0.0%	[0.0%, 14.9%]	0.0% to 0.0%
pill	26	0	0.0%	[0.0%, 12.9%]	0.0% to 0.0%
screw	41	0	0.0%	[0.0%, 8.6%]	0.0% to 0.0%
tile	33	0	0.0%	[0.0%, 10.4%]	0.0% to 0.0%
toothbrush	12	0	0.0%	[0.0%, 24.2%]	0.0% to 0.0%
transistor	60	0	0.0%	[0.0%, 6.0%]	0.0% to 0.0%
wood	19	0	0.0%	[0.0%, 16.8%]	0.0% to 0.0%
zipper	32	1	3.1%	[0.6%, 15.7%]	0.0% to 3.1%
all	467	7	1.5%	[0.7%, 3.1%]	

seeds. No other category exceeds 4.8% on any seed.

The intervals are wide because the test normals are few. With 12 test normals, **toothbrush** shows zero false alarms, and the upper end of its interval is still 24.2%. With 60 normals, **transistor** still has an upper end of 6.0%. A single false alarm in 32 normals, as in **leather** and **zipper**, is a rate of 3.1%, the smallest non-zero rate that a category of that size can show. So zero observed false alarms in one category is weak evidence that the 1% target is met, and one false alarm is weak evidence that it is missed. Only the pooled interval is narrow enough to say that the overall rate is in the low single percent.

Why carpet fails. On the MPS run, the threshold that would give a 1% false-alarm rate on the **carpet** test normals is 1.933, while the largest of all 280 calibration scores is 1.788. Every calibration image scores below the level that the test normals need. No rule that uses only

Table 6: Threshold rules on real data (seed 0, MPS), 1% target. q99 is the 99th percentile. FAR and recall are means over the 15 categories, measured on the test split.

rule	within target	mean FAR	mean recall
old (holdout, q99)	3 / 15	9.0%	93.3%
5-fold, q99	10 / 15	3.4%	87.4%
5-fold, tolerance	13 / 15	1.9%	79.4%

Table 7: Per-category view of Table 6: training normals (train), holdout size of the old rule (hold), test normals (test), false alarms under the old rule, the 5-fold 99th percentile (q99) and the 5-fold tolerance bound (tol.), and recall under the tolerance bound.

category	train	hold	test	false alarms			recall
				old	q99	tol.	
bottle	209	21	20	0	0	0	96.8%
cable	224	22	58	1	0	0	78.3%
capsule	219	22	23	2	0	0	40.4%
carpet	280	28	28	12	9	7	97.8%
grid	264	26	21	1	1	0	73.7%
hazelnut	391	39	40	0	0	0	98.6%
leather	245	24	32	5	2	0	100.0%
metal_nut	220	22	22	2	0	0	93.5%
pill	267	27	26	2	0	0	39.0%
screw	320	32	41	0	0	0	41.2%
tile	230	23	33	1	0	0	75.0%
toothbrush	60	6	12	3	0	0	76.7%
transistor	213	21	60	5	0	0	92.5%
wood	247	25	19	1	1	0	93.3%
zipper	240	24	32	1	1	1	95.0%

the training normals can reach the target here. We read this as a shift between the training and test normals of this category. We did not use test data to set any threshold.

5.5 Threshold calibration

Table 6 compares the three rules of Section 4 on the same features (seed 0, MPS). The old rule meets the 1% target in 3 of 15 categories, with a mean false-alarm rate of 9.0%. On **carpet** it flags 12 of 28 normals, 42.9%. 5-fold calibration with the 99th percentile meets the target in 10 of 15, and the tolerance bound in 13 of 15. The price is recall on defective images, which falls from 93.3% to 87.4% and then to 79.4%. The recall loss is concentrated in **capsule**, **pill** and **screw**, where recall at the shipped threshold is 40.4%, 39.0% and 41.2% (Table 7).

Why the old rule fails. With 21 to 39 holdout scores (6 for **toothbrush**), the empirical 99th percentile is close to the sample maximum. The maximum of n draws sits on average near the $n/(n+1)$ quantile, about the 96th percentile for $n = 28$, not the 99th. So the threshold is too low and the false-alarm rate is too high. 5-fold calibration gives every training image an out-of-bank score, 60 to 391 scores instead of 6 to 39, but the 99th per-

Table 8: Threshold rules on synthetic $\mathcal{N}(0, 1)$ calibration scores, 20000 runs, 1% target.

rule	n	mean FAR	runs above 1%
old (holdout, q99)	28	4.02%	83.9%
5-fold, q99	280	1.33%	64.8%
5-fold, tolerance	280	0.36%	6.3%

centile is still a point estimate that falls below the true quantile in a large share of runs.

Synthetic check. To separate the rules from the data, we draw calibration scores from $\mathcal{N}(0, 1)$, where the true false-alarm rate of any threshold is known exactly, and use the **carpet** sizes: $n = 28$ for the old rule and $n = 280$ for the 5-fold rules. Table 8 shows 20000 runs. The old rule has a mean false-alarm rate of 4.02% and misses the target in 83.9% of runs. The 5-fold 99th percentile has a mean of 1.33% and misses in 64.8%. The tolerance rule has a mean of 0.36% and misses in 6.3% of runs. Because $280 < 299$, the tolerance rule falls back to the sample maximum here, and the maximum of 280 draws exceeds the true 99th percentile with probability $0.99^{280} \approx 6.0\%$. The observed 6.3% is close to this value.

The guarantee is rarely available. The 95%-confidence, 99%-coverage bound needs at least 299 calibration images. Only **hazelnut** (391) and **screw** (320) have that many training normals. For the other 13 categories the shipped threshold is the calibration maximum and the guarantee is recorded as not met. Even for **hazelnut** and **screw**, the selected order statistic is the largest calibration score, rank 391 of 391 and 320 of 320.

5.6 Comparison with anomalib PatchCore

We ran anomalib 2.7.0’s `PatchcoreModel` on all 15 categories with seeds 0 to 4 on the same RTX A5000. Only the model is anomalib’s: its `timm WideResNet50-2` feature extractor, its `k-center coreset`, its image score with the paper’s reweighting, and its anomaly map with a Gaussian blur of $\sigma = 4$. The image files, the preprocessing, the masks and all metric code are ours, and anomalib’s training engine, pre-processor and evaluator are not used. Both use a 1% coreset.

Table 9 shows that the mean metrics of the two implementations are within 0.003 of each other. The largest per-category gap is **screw** image AUROC, 0.9438 for ours against 0.9720 for anomalib. This is consistent with the missing score reweighting, which mainly affects the image score, but the two implementations also differ in feature extractor, coreset code and map smoothing, so the comparison does not isolate the reweighting. For

AUPRO, anomalib leads on **grid** by 0.0209 and ours leads on **cable** by 0.0260.

5.7 Device dependence

The single-seed MPS run and the CUDA run with seed 0 use the same script and the same seed. They do not give the same numbers. Table 10 shows **carpet**, the category where this matters most. The calibrated threshold moves from 1.788 to 1.824, and the number of false alarms among the 28 test normals moves from 7 to 5. The Wilson intervals of the two counts, 12.7% to 43.4% and 7.9% to 35.6%, overlap widely, so the device effect is small compared with the sampling uncertainty, but it is large enough to change a reported count. We therefore keep MPS and CUDA results in separate tables. We did not record whether library versions also differed between the two runs, so we cannot attribute the whole difference to the device.

6 Discussion

Evaluation code changes the result as much as the model. Fixing the pixel-metric protocol moved the mean AUPRO from 0.8457 to 0.9025. The gap between our implementation and anomalib’s, measured with shared metric code, is 0.0007 in mean AUPRO. A comparison between methods that does not share metric code can therefore be dominated by the metric implementation.

Seeds matter at the third decimal. On **screw**, **grid** and **capsule** the seed standard deviation of image AUROC is between 0.0072 and 0.0175. Per-category comparisons at that resolution need several seeds.

A pooled false-alarm rate can hide a failing category. The pooled rate of 1.5% looks acceptable, while **carpet** sits near 18%. With MVTec AD’s small test sets, per-category false-alarm claims should come with intervals, and a zero count should not be read as meeting the target.

A threshold rule should be judged by how often it misses. The 99th percentile is unbiased in a loose sense, but a deployment target is one-sided. The tolerance bound states the claim in the form a deployment needs, at least 99% of normal parts below the threshold with 95% confidence, and its sample-size requirement of 299 images is a concrete requirement for data collection. On MVTec AD this requirement is met in only 2 of 15 categories.

Detection is not localisation. Several categories detect well but localise poorly, and a supervised classifier

Table 9: This repository against anomalib 2.7.0 PatchCore, same images, preprocessing, masks, seeds (0 to 4), metric code and GPU. Per-category values are means over seeds; “diff” is ours minus anomalib. The last row is the mean over categories, then mean $\pm$ sample standard deviation over seeds.

category	image AUROC			pixel AUROC			AUPRO		
	ours	anomalib	diff	ours	anomalib	diff	ours	anomalib	diff
bottle	1.0000	1.0000	+0.0000	0.9831	0.9806	+0.0025	0.9291	0.9232	+0.0059
cable	0.9968	0.9871	+0.0097	0.9841	0.9780	+0.0061	0.9244	0.8984	+0.0260
capsule	0.9777	0.9777	+0.0000	0.9867	0.9873	−0.0006	0.9075	0.9090	−0.0015
carpet	0.9887	0.9855	+0.0032	0.9878	0.9877	+0.0000	0.9313	0.9290	+0.0022
grid	0.9781	0.9784	−0.0003	0.9700	0.9751	−0.0051	0.8780	0.8988	−0.0209
hazelnut	1.0000	1.0000	+0.0000	0.9844	0.9834	+0.0009	0.9365	0.9338	+0.0026
leather	1.0000	1.0000	+0.0000	0.9906	0.9895	+0.0011	0.9584	0.9654	−0.0070
metal_nut	0.9988	0.9997	−0.0009	0.9844	0.9806	+0.0038	0.9157	0.9179	−0.0021
pill	0.9541	0.9578	−0.0037	0.9766	0.9752	+0.0014	0.9284	0.9317	−0.0033
screw	0.9438	0.9720	−0.0282	0.9786	0.9796	−0.0010	0.9060	0.9105	−0.0045
tile	0.9879	0.9895	−0.0016	0.9552	0.9483	+0.0069	0.7953	0.7931	+0.0022
toothbrush	0.9967	0.9933	+0.0033	0.9856	0.9852	+0.0005	0.8423	0.8407	+0.0016
transistor	0.9997	0.9994	+0.0002	0.9732	0.9653	+0.0079	0.9463	0.9409	+0.0054
wood	0.9912	0.9905	+0.0007	0.9398	0.9298	+0.0100	0.8344	0.8423	−0.0079
zipper	0.9960	0.9921	+0.0039	0.9826	0.9787	+0.0040	0.9282	0.9171	+0.0111
mean, ours: 0.9873 $\pm$ 0.0016, 0.9775 $\pm$ 0.0003, 0.9041 $\pm$ 0.0005									
mean, anomalib: 0.9882 $\pm$ 0.0013, 0.9750 $\pm$ 0.0003, 0.9035 $\pm$ 0.0009									
mean diff image AUROC −0.0009, pixel AUROC +0.0026, AUPRO +0.0007									

Table 10: *carpet*, seed 0, same script on an Apple M4 (MPS) and an RTX A5000 (CUDA). Threshold is the shipped 5-fold tolerance-bound threshold; false alarms are out of 28 test normals.

	MPS	CUDA
threshold	1.788	1.824
false alarms	7 / 28 (25.0%)	5 / 28 (17.9%)
Wilson 95%	[12.7%, 43.4%]	[7.9%, 35.6%]
recall	97.8%	96.6%
image AUROC	0.9904	0.9880
pixel AUROC	0.9880	0.9878
AUPRO	0.9333	0.9307

can detect well with an explanation map that does not point at the defect. Localisation claims need a control.

7 Limitations

- **One dataset.** All results are on MVTec AD. We do not evaluate on MVTec AD 2 [7] or on other industrial datasets, and MVTec AD is close to saturated for image AUROC.
- **One backbone and one coreset size.** We use WideResNet50-2 with `layer2` and `layer3` and a 1% coreset for all headline results. The ablations cover only `screw` and predate the metric fix.
- **Missing reweighting.** Our image score omits PatchCore’s reweighting, so our numbers are for a simplified PatchCore.
- **Few test normals.** Per-category false-alarm rates rest on 12 to 60 normal images, and their intervals

are wide. A tighter estimate needs more normal test data than the benchmark provides.

- **Calibration sizes.** 13 of 15 categories have fewer than the 299 calibration images the tolerance guarantee needs.
- **Mixed devices.** Single-seed, threshold-rule and localisation results were run on MPS, and the seed and anomalib results on CUDA. Section 5.7 shows that this changes some numbers.
- **Data source.** The images come from a public mirror of MVTec AD, not from the official download.
- **Published values.** The “paper” columns are the values recorded in our repository. Seed counts and protocol details of the published runs may differ from ours.
- **Supervised baseline.** The classifier comparison covers three categories, one seed and a 50/50 split of the defective test images.

8 Reproducibility statement

Code, result files and figures are public at <https://github.com/aghasalim/explainable-defect-detector>, archived at doi:10.5281/zenodo.23003629. All numbers in this paper come from the repository at commit

aba8264c6c8bca6e2f4be8fb4cfedd85e5e86d38

The main entry points are `src/edd/sweep.py` (single seed), `src/edd/seeds.py` (five seeds and

false-alarm intervals), `src/edd/threshold_bug.py` (threshold rules, with `--synthetic` for Table 8), `src/edd/vs_anomalib.py` (anomalib comparison) and `src/edd/explain.py` (localisation). Raw per-run results are in `reports/`, including `reports/seeds/` and `reports/anomalib/`. The test suite runs without data in continuous integration and includes known-answer tests for the pixel metrics, the Wilson interval and the synthetic threshold experiment. A separate verification script rebuilds the published tables from the raw result files in other languages, including the false-alarm intervals with R’s `prop.test`. The figures in this paper are drawn directly from the committed JSON files. The code is MIT licensed. MVTec AD is licensed CC BY-NC-SA 4.0.

References

- [1] Samet Akcay, Dick Ameln, Ashwin Vaidya, Barath Lakshmanan, Nilesch Ahuja, and Utku Genc. Anomalib: A deep learning library for anomaly detection. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 1706-1710, 2022. doi: 10.1109/ICIP46576.2022.9897283.
- [2] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTec AD: A comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9592-9600, 2019.
- [3] Paul Bergmann, Kilian Batzner, Michael Fauser, David Sattlegger, and Carsten Steger. The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision*, 129(4):1038-1059, 2021. doi: 10.1007/s11263-020-01400-4.
- [4] Niv Cohen and Yedid Hoshen. Sub-image anomaly detection with deep pyramid correspondences. *arXiv preprint arXiv:2005.02357*, 2020.
- [5] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. PaDiM: A patch distribution modeling framework for anomaly detection and localization. In *Pattern Recognition. ICPR International Workshops and Challenges*, volume 12664 of *Lecture Notes in Computer Science*, pages 475-489. Springer, 2021.
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770-778, 2016.
- [7] Lars Heckler-Kram, Jan-Hendrik Neudeck, Ulla Scheler, Rebecca König, and Carsten Steger. The MVTec AD 2 dataset: Advanced scenarios for unsupervised anomaly detection. *arXiv preprint arXiv:2503.21622*, 2025.
- [8] Aghasolim Mustafazada. Explainable visual defect detector. Software, <https://github.com/aghasolim/explainable-defect-detector>, 2026.
- [9] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14318-14328, 2022.
- [10] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618-626, 2017.
- [11] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations (ICLR)*, 2018.
- [12] S. S. Wilks. Determination of sample sizes for setting tolerance limits. *The Annals of Mathematical Statistics*, 12(1):91-96, 1941.
- [13] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209-212, 1927. doi: 10.1080/01621459.1927.10502953.
- [14] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2016.